

Tic-Tac-Toe game

Code:

```
board = [' ' for _ in range(9)]
```

```
def print_board(board):
```

```
    print(f'\n {board[0]} | {board[1]} | {board[2]} ')
```

```
    print("-----")
```

```
    print(f' {board[3]} | {board[4]} | {board[5]} ')
```

```
    print("-----")
```

```
    print(f' {board[6]} | {board[7]} | {board[8]} \n')
```

```
def check_winner(b, player):
```

```
    win_conditions = [
```

```
        [0,1,2], [3,4,5], [6,7,8],
```

```
        [0,3,6], [1,4,7], [2,5,8],
```

```
        [0,4,8], [2,4,6]
```

```
    ]
```

```
    return any(all(b[i] == player for i in cond) for cond in win_conditions)
```

```
def is_full(b):
```

```
    return ' ' not in b
```

```
def minimax(b, is_maximizing):
```

```
    if check_winner(b, 'O'): return 1
```

```
    if check_winner(b, 'X'): return -1
```

```
    if is_full(b): return 0
```

```
    if is_maximizing:
```

```
        best_score = -float('inf')
```

```
        for i in range(9):
```

```

        if b[i] == ' ':
            b[i] = 'O'

            score = minimax(b, False)

            b[i] = ' '

            best_score = max(score, best_score)

    return best_score

else:
    best_score = float('inf')

    for i in range(9):
        if b[i] == ' ':
            b[i] = 'X'

            score = minimax(b, True)

            b[i] = ' '

            best_score = min(score, best_score)

    return best_score

```

```

def ai_move(b):
    best_score = -float('inf')
    move = None

    for i in range(9):
        if b[i] == ' ':
            b[i] = 'O'

            score = minimax(b, False)

            b[i] = ' '

            if score > best_score:
                best_score = score
                move = i

    b[move] = 'O'

```

```
print("You are X, AI is O.")
```

```
print_board(board)
```

```
while True:
```

```
    pos = int(input("Enter move position (0-8): "))
```

```
    if board[pos] == ' ':
```

```
        board[pos] = 'X'
```

```
    else:
```

```
        print("Invalid move! Try again.")
```

```
        continue
```

```
    if check_winner(board, 'X'):
```

```
        print_board(board)
```

```
        print("You win!")
```

```
        break
```

```
    if is_full(board):
```

```
        print_board(board)
```

```
        print("It's a tie!")
```

```
        break
```

```
    ai_move(board)
```

```
    print("AI played:")
```

```
    print_board(board)
```

```
    if check_winner(board, 'O'):
```

```
        print("AI wins!")
```

```
        break
```

```
    if is_full(board):
```

```
print("It's a tie!")
```

```
break
```

Output:

You are X, AI is O.

```
|  |
-----
|  |
-----
|  |
```

Enter move position (0-8): 8

AI played:

```
|  |
-----
| O |
-----
|  | X
```

Enter move position (0-8): 5

AI played:

```
|  | O
-----
| O | X
-----
|  | X
```

Enter move position (0-8): 6

AI played:

```
|  | O
-----
| O | X
-----
X | O | X
```

Enter move position (0-8): 1

AI played:

```
O | X | O
-----
| O | X
-----
X | O | X
```

Enter move position (0-8): 3

```
O | X | O
-----
X | O | X
-----
X | O | X
```

It's a tie!

Vacuum cleaner agent

Code:

```
class VacuumCleanerAgent:
    def __init__(self):
        self.location = "A"

    def act(self, environment):
        status = environment[self.location]

        if status == "Dirty":
            print(f"Location {self.location} is dirty -> Suck")
            environment[self.location] = "Clean"

        elif self.location == "A":
            print("Location A is clean -> Move Right")
            self.location = "B"

        else:
            print("Location B is clean -> Move Left")
            self.location = "A"

# Take input from user
a = input("Is location A dirty or clean? ").capitalize()
b = input("Is location B dirty or clean? ").capitalize()

environment = {
    "A": a,
    "B": b
}
```

```
agent = VacuumCleanerAgent()

print("\n--- Vacuum Cleaner Agent ---")

for _ in range(4):
    agent.act(environment)
    print("Environment:", environment)
```

Output:

```
Is location A dirty or clean? dirty
Is location B dirty or clean? clean

--- Vacuum Cleaner Agent ---
Location A is dirty -> Suck
Environment: {'A': 'Clean', 'B': 'Clean'}
Location A is clean -> Move Right
Environment: {'A': 'Clean', 'B': 'Clean'}
Location B is clean -> Move Left
Environment: {'A': 'Clean', 'B': 'Clean'}
Location A is clean -> Move Right
Environment: {'A': 'Clean', 'B': 'Clean'}
```